

Core Banking System on Azure – Design Document

Version: 1.0 | Owners: Architecture Team | License: MIT (original work)

1. Purpose & Scope

Design a secure, scalable core banking platform supporting retail accounts, payments, internal transfers, statements, and regulatory-grade auditability. The solution is built on Microsoft Azure using a React single-page application (SPA), .NET Core Web APIs, and Azure SQL Database.

2. System Requirements

Functional Requirements

- Customer onboarding (KYC/AML checks) and account lifecycle management.
- Current/Savings accounts with real-time balance updates.
- Internal transfers, external payments initiation (FPS/SEPA-ready).
- Statements generation (monthly) and on-demand exports.
- Back-office portal for operations and support.
- Comprehensive audit trail for all financial operations.

Non-Functional Requirements

Attribute	Target
Availability	99.95% single-region; 99.99% with active-passive DR
Latency	P99 < 200 ms read; P99 < 400 ms write (in-region)
Throughput	2k TPS peak (scalable to 10k+)
Security & Privacy	GDPR/UK DPA; encryption in transit/at rest; least privilege
Auditability	Immutable audit log; tamper-evident history
Observability	100% API tracing; structured logs; SLO dashboards
Cost	PaaS-first; autoscale; retention policies for logs

3. High-Level Architecture

The system follows a layered, service-oriented architecture. The React SPA authenticates against Entra ID (OIDC, PKCE) and calls .NET Core APIs via Azure API Management (APIM). APIs run on Azure App Service behind Application Gateway (WAF) and Azure Front Door for global routing. Data persists in Azure SQL Database (Hyperscale); hot reads are cached in Azure Redis. Asynchronous processing uses Azure Service Bus. Secrets are stored in Azure Key Vault; private endpoints secure PaaS egress.

Layer/Service	Azure Service / Tech	Purpose
Client UI	React SPA (Static Web Apps or App Service)	User interactions, forms, dashboards
Edge	Front Door + WAF	Global routing, TLS, bot/DDoS protection
Gateway	API Management	AuthN/AuthZ, rate limits, API versioning, policies

Compute	.NET Core Web API on App Service (Linux)	Business services, domain logic, controllers
Data	Azure SQL (Hyperscale)	OLTP ledger, accounts, transactions, audit
Cache	Azure Cache for Redis	Hot keys, session tokens (short TTL)
Events	Azure Service Bus (Premium)	Outbox → consumers (notifications, statements)
Storage	Azure Blob Storage	Statements, exports, attachments
Secrets	Azure Key Vault + Managed Identity	Keys, connection strings, certs
Observability	App Insights + Log Analytics	Metrics, traces, logs, dashboards

4. Domain & Data Model

Key entities include Customer, Account, Transaction, LedgerEntry, and AuditEvent. A double-entry ledger ensures consistency. Use Azure SQL features such as Row-Level Security (RLS), Dynamic Data Masking, and Always Encrypted for sensitive fields.

Representative Tables

- Customer(CustomerId, ExternalRef, FirstName, LastName, Email, Phone, NationalId [Always Encrypted], CreatedAt)
- Account(AccountId, CustomerId, AccountNumber [unique], Type, Currency, Status, Balance, RowVersion)
- Transaction(TxnId, FromAccountId, ToAccountId, Amount, Currency, Status, Channel, RequestedAt, PostedAt)
- LedgerEntry(LedgerEntryId, AccountId, TxnId, Direction[D/C], Amount, Currency, Narration, BookedAt, CreatedAt)
- AuditEvent(AuditId, Principal, Action, Resource, HashPrev, HashCurr, CreatedAt)

5. API Design (.NET Core)

Authentication uses Entra ID with OAuth2/OIDC (Auth Code + PKCE). The SPA obtains an access token and calls APIM, which enforces policies before routing to backend APIs. REST conventions with versioning (/v1) and idempotency for sensitive operations (Idempotency-Key header) are used.

Representative Endpoints

- GET /v1/customers/{id} • POST /v1/customers
- GET /v1/accounts/{id} • POST /v1/accounts • GET /v1/customers/{id}/accounts
- POST /v1/transactions/transfer • GET /v1/transactions/{txnId}
- GET /v1/accounts/{id}/transactions?from=YYYY-MM-DD&to;=YYYY-MM-DD
- GET /v1/statements/{accountId}?period=YYYY-MM • POST /v1/payments/fps

6. Posting & Consistency Model

Transfers execute within a serializable transaction to guarantee atomic debit/credit posting. Balance updates use optimistic concurrency via ROWVERSION. An outbox table ensures reliable event publication to Service Bus. Read paths favor cache for hot balances; cache is invalidated on commit.

7. Security & Compliance

- Identity: Entra ID; SPA uses MSAL; backend validates JWT (aud/iss).
- Authorization: Role-based policies; least-privilege RBAC.
- Data protection: TDE for SQL; Always Encrypted for NI/PAN; TLS 1.2+ end-to-end.
- Network: Private Endpoints for SQL/Storage/Key Vault; NSGs; Azure Firewall.
- Threat protection: WAF (OWASP), rate limiting (APIM), DDoS Standard; Defender for Cloud/SQL/Storage.
- Privacy/GDPR: Data minimization, retention by product, subject rights workflows.
- Secrets: Managed Identity; no secrets in code or pipeline vars.

8. Resilience, Backup & DR

Azure SQL geo-replication to paired region; point-in-time restore; weekly full/daily diff/5-min log backups. Front Door + APIM failover profiles. Quarterly DR drills with documented runbooks.

9. Observability

- Distributed tracing (W3C TraceContext) across SPA→APIM→API→SQL.
- Structured logging (Serilog) with correlation IDs; PII scrubbing.
- SLO dashboards: TPS, p50/p90/p99 latency, error rates, queue DLQs.
- Synthetics and health probes; alerting with actionable runbooks.

10. Scaling & Performance

Stateless APIs scale horizontally via App Service plans; SQL scales via vCores and Hyperscale tier. Use read replicas for reporting, partition large tables, and consider sharding by AccountId hash for hotspots. Front Door caching and Redis reduce origin load.

11. CI/CD & Environments

GitHub Actions or Azure DevOps pipelines: build, unit tests, SAST (CodeQL), dependency scan (Dependabot), IaC scan, blue/green slots, infra provisioning (Bicep/Terraform). Database migrations via DACPAC/Flyway. Secrets pulled at deploy via Managed Identity.

12. Risks & Mitigations

Risk	Mitigation
Hot row contention on popular accounts	Adopt sharding/partitioning; batching; retry with back-off; narrow transactions
Eventual consistency side-effects	Outbox pattern; idempotent consumers; de-duplication keys.
Vendor lock-in	Abstract data access; open standards (OIDC/REST); IaC to enable portability
Regulatory change	Policy engine (OPA); feature flags; regular compliance reviews.

13. Appendix – Representative API Routes & Policies

- Idempotency: Clients send Idempotency-Key; APIM enforces uniqueness per consumer/route; backend stores key→txn mapping.
- Versioning: /v1, /v2 with deprecation headers.
- Error Handling: RFC7807 problem+json; correlation IDs in responses.
- Rate Limits: Tiered per client; burst + sustained limits.

Architecture Diagram (conceptual)

[Browser/React SPA] → [Front Door/WAF] → [API Management] → [.NET Core APIs on App Service] → [Azure SQL]
■ [Redis] ■ [Service Bus] ■ [Blob Storage]
Secrets via Managed Identity ↔ Key Vault | Private Endpoints for SQL/Storage/APIM